

User Manual

Complete reference for all twelve skills. Written against **gt v0.9.0**.

The model

Claude forgets everything between sessions. The vault doesn't. Golden Thread's job is to move what you learn *out* of a session into a file whose **location** tells you how widely it applies — then load only the slice the next session needs.

```

session conversation
  ↓ gt-work
Projects/<slug>/memory/*.md      this project, this detail
  ↓ gt-promote
research.md                      dated findings — `unverified` allowed, if labelled
  ↓ gt-promote + gt-validate
decisions.md · design.md         settled choices — independently verified
  ↓ gt-promote + gt-validate
Knowledge/                       true for several projects — independently verified
  ↓ gt-promote + gt-validate
global-memory/                   true everywhere — independently verified

```

Facts move **up** as they prove general. They never move back down, and nothing is silently deleted.

Verification gates the climb — it is not a step at the end. `research.md` holds dated findings and accepts an `unverified` figure so long as it is labelled. Every level above it *reads as settled*, so a figure entering `decisions.md`, `design.md`, `Knowledge/` or `global-memory/` — or driving a production change — must be **independently verified**: re-derived by a validator that never saw the reasoning. A second pass by the same session is only `self-verified`; it shares the assumptions that produced the error, so it confirms rather than checks. Run `/gt:gt-validate` *before* the promotion.

This rung was added after an unverified analysis reached `decisions.md` as an ADR recommending a four-ticker production change. An independent validator refuted three of the four — two of which would have deepened portfolio drawdown while appearing to improve their own ticker's. The ladder carried the error upward because nothing on it asked how the number was checked.

The discipline: write a fact at the narrowest scope that is honest, and promote it only when a second project proves it general. A fact promoted too early becomes a rule you must remember to disbelieve.

Loading is lazy, by design

`gt-open` reads the project's core docs and then reads `memory/MEMORY.md` **only** — an index of one line per file. The memory files themselves are read on demand.

A project with 70 notes costs ~80 lines to open instead of ~2,000. This is why `MEMORY.md` descriptions matter: they are the entire basis for deciding whether a file is worth opening.

Vault layout

```
<vault>/
  CLAUDE.md           Knowledge page conventions and schema
  index.md            navigational index of Knowledge pages
  log.md              audit trail
  review-queue.md     items flagged for owner review
  lint-declines.md    suppressions, each with its reason

  Sources/           IMMUTABLE raw originals
  Knowledge/          cross-project wiki pages
  global-memory/      loaded in every session
    MEMORY.md         the index – this is what actually gets read

  Projects/
    README.md         master list + Dataview views
    CONVENTIONS.md    stages, properties, domain taxonomy, file roles
    PROTOCOL.md       cross-project process rules
    INFRASTRUCTURE.md the server fleet, defined ONCE

<slug>/
  README.md          status board + property frontmatter
  source.md          where the code lives + deploy plan
  idea.md            original brain dump – IMMUTABLE
  research.md        append-only dated findings
  decisions.md       append-only numbered ADRs
  design.md          current architecture
  spec.md            handoff artifact (when design is complete)
  runbook.md         operational procedures (optional)
  memory/
    MEMORY.md        index – the only thing gt-open reads
    <topic>.md        loaded on demand
```

Project properties

Every project `README.md` carries frontmatter. These are real Obsidian properties — they drive the tag pane, search, and the Dataview views.

```
---
type: project
slug: my-project          # must match the folder name
domain: trading           # coarse grouping
stage: active             # idea|research|design|active|complete|archived
topology: bastion-direct  # local|remote|bastion-jump|bastion-direct|n/a
tags: [trading, platform, live]
parent: other-slug        # sub-projects only
---
```

Categorise with properties, not folders. Do not add a category level under `Projects/`. `gt-lint` walks projects to find `memory/` directories; a category folder is fine for *that* check since v0.3.0, but folders also fragment `gt-open`'s project lookup and buy nothing Obsidian doesn't already give you through properties. The one legitimate second level is a real sub-project created with `--parent`.

Source topology

`source.md` records **where the code actually lives**. It is read first by `gt-open`, because acting without it is how you edit the wrong box.

TOPOLOGY	MEANING
<code>local</code>	A folder on this machine. No deploy step.
<code>remote</code>	One repo, one server. Repo mirrors the prod structure; the server pulls from GitHub.
<code>bastion-jump</code>	Several servers, all reached through one gateway host.
<code>bastion-direct</code>	Several servers, each reachable independently.

Addressing is **role × env → host + path**, because one machine can serve several vhosts and the same role exists in more than one environment.

Bastion topologies also carry a **file plan**, marking each deployed file:

- **static** — byte-identical on every target. Deploy from one canonical copy.
- **unique** — a per-server variant. Never copy one host's version over another's.

That distinction is the whole point. Two files with the same name on two hosts may be the same file that must not drift, or different files that must not be merged — and nothing but the file plan tells you which.

The fleet itself is defined **once** in `Projects/INFRASTRUCTURE.md` and linked from each `source.md`. Never copy the host table into a project.

Typical use cases

Recipes for the situations that actually recur. Each names the command; the per-command detail follows in the sections below.

Starting something new

```
/gt:gt-create → work → /gt:gt-work
```

`gt-create` gathers slug, domain, topology and tags, scaffolds the folder, and writes `idea.md` from what you actually said. `idea.md` is **immutable after that** — it is the traceable "why", and the one file you never rewrite when the plan changes. Close the session with `gt-work` or the session's findings die with it.

Picking up a project after time away

```
/gt:gt-open <slug>
```

Reads the project docs in dependency order and stops at `memory/MEMORY.md`. Skim the returned summary for **stage**, **hosts** and **blockers** before touching anything — a stale host entry is cheapest to correct before work starts. Then pull individual memory notes by name as the work needs them.

"What should I work on?"

```
python3 <vault>/Projects/golden-thread/tools/gt_tasks.py # then read TASKS.md
```

Regenerate first, always. Effective priority is computed against the clock — a `p:: 1` older than seven days escalates, a `due` inside three days escalates, an open `pp_escalate` window escalates. Reading a stale `TASKS.md` gives you last week's ranking with this week's confidence.

`waiting:: user` is your list; `waiting:: agent` is the session's.

Bringing an existing project into the vault

```
/gt:gt-ingest
```

Copies — never moves or deletes. If the notes cross-reference each other by `[[wikilink]]`, **count the links before renaming anything**; Obsidian resolves links by filename, so a tidy-up destroys the graph. Prefer keeping filenames.

You just learned something – where does it go?

```
/gt:gt-work          (end of session)
/gt:gt-promote        (when it proves general)
```

File at the **narrowest scope that is honest**. A finding that stopped changing graduates from `memory/` to `research.md`; a truth a *second* project hit graduates to `Knowledge/`. See "Where does this fact go?" below. Promotion is cheap; demotion is not.

A number is about to be recorded as fact

```
/gt:gt-validate
```

Before it lands in `decisions.md`, `design.md`, `Knowledge/` or `global-memory/`, and before any production change. The validator gets the claim, the rules and the artifact — never your reasoning. Budget for the possibility that it comes back **refuted**; that is the skill working, not failing.

Answering "how does this work?"

```
/gt:gt-query <topic>
```

Reads `index.md`, follows wikilinks into `Knowledge/`, then falls back to grep and project memory. If a page comes back `status: stale`, verify before acting on it.

Keeping the vault honest

```
/gt:gt-lint           # broken links, orphans, unlisted memory, scope leaks
/gt:gt-runbook-lint    # facts duplicated across runbooks
/gt:gt-refresh         # upstream changes to Sources/
```

Run `gt-lint` after any structural change. Triage into three piles: *you broke it* (fix now), *already broken* (record in `review-queue.md`), *false positive* (suppress **with the reason**). The check that matters most is `memory-unlisted` — a memory file missing from `MEMORY.md` is a file Claude will never load.

Capturing what you jotted down elsewhere

```
/gt:gt-review
```

Scans Obsidian daily notes for uncaptured tasks and ideas and promotes the ones you pick into tracked projects.

Setup

`/gt:gt-init`

First run, or wiring a new machine. Writes `~/.claude/vault-config.json` (the pointer every other skill reads), scaffolds the vault, and adds a Golden Thread section to `~/.claude/CLAUDE.md`. Idempotent.

`CLAUDE.md` is read at **session start**. A rule added mid-session does not apply until you restart — that looks like the rule being ignored.

`/gt:gt-create`

Scaffolds a project. Gathers slug, title, tags, **domain**, sub-project parent, runbook, and **topology**; runs the script; then fills `idea.md` from what you actually said. `idea.md` is immutable afterwards — it is the traceable "why".

Daily work

`/gt:gt-open <slug>`

Loads a project and stops. It is **read-only** — nothing is written during loading.

Reads, in order: `CONVENTIONS.md` and `PROTOCOL.md` (once per session), the project `README.md` — frontmatter first, since `domain / stage / topology / tags` is the fastest read of where the project stands — then `source.md` → `idea.md` → `research.md` → `decisions.md` → `design.md` → `spec.md` → `runbook.md`, the linked fleet page, and finally `memory/MEMORY.md` — **the index only**. Individual memory files load on demand.

`source.md` **leads the project docs deliberately**. It says which box serves which role in which environment. Acting before reading it is how you edit the wrong host, or overwrite a file that exists on three machines.

`research.md` **over ~200 lines is read by its headings**, plus the entries relevant to the work at hand and the most recent few. It is append-only and grows without bound; reading it whole crowds out the context the actual work needs.

Sub-projects load only when you name one, or the status board shows it as the active item.

Announces the `review-queue.md` count once, then summarises stage, **topology and hosts**, next action, blockers, and what memory exists but is unloaded — the host list so you can correct a stale entry *before* work starts rather than after.

`/gt:gt-work`

End of session. The step people skip, and skipping it is what makes the vault decay.

GOES TO	WHAT BELONGS THERE
research.md	Dated findings and gotchas. New discoveries only.
decisions.md	Numbered ADR: decision, context, rejected alternatives. Stable choices only.
design.md	Rewritten in place if architecture changed. Describes <i>now</i> .
memory/*.md	Session state, updated in place.
spec.md	Created when design is complete enough to hand off.

Also updates the `stage:` property when the project changes phase.

Not in `decisions.md`: anything you might change next session. An ADR you reverse next week teaches the vault to lie to you.

/gt:gt-query

Reads `index.md`, follows wikilinks 2–3 hops into `Knowledge/`, falls back to grep, then project memory. Be specific. If a result is `status: stale`, verify it.

/gt:gt-ingest

Bulk-imports an existing project. Copies — never moves or deletes.

The trap: if the notes cross-reference each other by `[[wikilink]]`, renaming or merging files destroys that graph, because Obsidian resolves links by filename. Measure first:

```
grep -roh '\[([^\]]*\]\)' --include='*.md' . | wc -l
```

Prefer keeping filenames. If you must repoint, match the *whole* link — `[[name]]` and `[[name|alias]]` — never a substring, or `[[foo]]` will corrupt `[[foo_bar]]`. Never repoint links inside `Sources/`.

/gt:gt-review

Scans Obsidian daily notes for uncaptured tasks and ideas, then promotes selected ones into tracked projects.

Knowledge management

/gt:gt-promote

The judgement call: **has a second project proved this general?**

FROM → TO	WHEN
memory/ → research / decisions	The finding stopped changing session to session
project → Knowledge/	A second project hit the same truth
Knowledge/ → global-memory/	Needed in <i>every</i> session, <i>every</i> project
anywhere → new project	The idea doesn't belong where it is

Nothing is deleted. Retiring sets `status: stale` and records what superseded it.

`/gt:gt-refresh`

Checks `Sources/` for upstream changes. Supersedes with a **new** immutable file carrying `supersedes:` — never edits the old one.

Verification

`/gt:gt-validate`

Verify a claim by **re-deriving** it, not by reviewing it. Use before a finding is recorded as fact, before a production change, or whenever a number matters.

The one rule: the validator never receives the reasoning that produced the claim. Give it the reasoning and it grades the argument — inheriting the same blind spot. Give it only the claim, the rules and the artifact, and it has to go back to primary sources. That is the only thing that catches a wrong premise.

Reduce the work to a **single falsifiable assertion**. "The analysis is sound" is not validatable; "widening NQ's target to 1.25 improves risk-adjusted return" is. Validate several claims **separately** — a bundled claim returns a bundled verdict, which hides which part failed.

Loads the project's `validation-rules.md`, and the parent's too, since packs are inherited. Those are standing invariants the validator enforces whether or not the request mentions them — because *a requester who has already made a domain error will not think to ask the validator to check for it*. On conflict, the pack wins.

CLASS	USE WHEN THE RISK IS
<code>empirical</code>	A number, measurement or result could be wrong
<code>vantage</code>	The measuring position may not be able to observe the answer
<code>rule-compliance</code>	Work must satisfy <code>core-rules/</code> or <code>CONVENTIONS.md</code>
<code>code</code>	Code may not do what its name, comment or docs claim

Pick every class whose risk is present. **Do not substitute one generic reviewer** — that produces agreement, not verification.

The packet is exactly three fields — `claim` , `rules` , `artifact` — then re-read it and strip your conclusions, your numbers, your confidence, and any adjective implying the expected answer. "Confirm that X" becomes "determine whether X". Withholding *reasoning* is isolation; withholding *access* is just a broken validator, so keep the hostnames, API shapes and filters it needs to reach the artifact alone.

Three verdicts: **confirmed**, **refuted** (quantify the divergence), and **cannot-verify** (name what was missing).

cannot-verify is never a pass. A validator that could not check something and stayed quiet manufactures false assurance — worse than no validator at all.

You are the worst possible author of this packet, because you already know the answer and your framing leaks. Treat building it as an adversarial exercise against yourself. A validator disagreeing is a *result*, not a failure.

Maintenance

`/gt:gt-lint`

CHECK	CATCHES
<code>index-gap</code>	Knowledge page not in <code>index.md</code> — <code>gt-query</code> won't find it
<code>broken-link</code>	<code>[[wikilink]]</code> resolving to nothing
<code>orphan</code>	Knowledge page linked from nowhere
<code>memory-unlisted</code>	<code>memory/*.md</code> not in <code>MEMORY.md</code> — Claude will never load it
<code>global-gap</code>	<code>global-memory/*.md</code> not in its index
<code>memory-bloat</code>	<code>global-memory</code> file over 30 lines
<code>global-scope-leak</code>	Project-specific content in <code>global-memory/</code>
<code>superseded-cited</code>	Knowledge page citing a superseded source
<code>stale</code>	Knowledge page marked <code>status: stale</code>
<code>source-todo</code>	<code>source.md</code> with no topology or deployment targets
<code>frontmatter</code>	Project README missing properties, or slug $\neq$ folder

Declines go in `lint-declines.md` as `suppress:` **with the reason**.

Triage findings into three piles: *you broke it* (fix now), *already broken* (record in `review-queue.md` — don't guess at a target), *false positive* (suppress with reason).

`/gt:gt-runbook-lint`

Finds procedures duplicated across runbooks and routes them to the right shared layer — `PROTOCOL.md` , a Knowledge page, or a repo `CLAUDE.md` .

Where does this fact go?

THE FACT	GOES TO
"This preset is wrong right now"	<code>memory/</code>
"This API rejects plural keys"	<code>research.md</code>
"We use POST for all mutations, and why"	<code>decisions.md</code>
"The system is structured like this now"	<code>design.md</code>
"This platform behaviour bit two projects"	<code>Knowledge/</code>
"Always diff before overwriting prod"	<code>global-memory/</code>
"How to restart the prod backtest"	<code>runbook.md</code>
"Never scp without backing up"	<code>PROTOCOL.md</code>
"This code lives on host X at path Y"	<code>source.md</code>
"Which box serves which role"	<code>INFRASTRUCTURE.md</code>

Two failure modes, opposite directions: **too narrow** buries a platform truth in one project, so the next rediscovers it the hard way; **too wide** promotes a quirk into `global-memory/` , where it loads into unrelated sessions as a rule that isn't true there. When unsure, file narrow — promotion is cheap, demotion isn't.

Script reference

```
SCRIPTS=~/.claude/plugins/cache/golden-thread-plugin/gt/0.9.0/scripts

python3 $SCRIPTS/vault_init.py fresh --vault ~/my-vault --domain "My Team"

python3 $SCRIPTS/vault_init.py create-project --vault ~/my-vault \
  --name my-project --title "My Project" --tags "backend,api" \
  --domain platform --topology bastion-direct \
  --repo-url git@github.com:me/my-project.git --runbook

python3 $SCRIPTS/vault_init.py create-project --vault ~/my-vault \
  --name sub-feature --parent my-project --domain platform

python3 $SCRIPTS/vault_init.py connect --vault ~/existing-vault

python3 $SCRIPTS/gt_ingest.py ~/Projects/my-project --json

python3 $SCRIPTS/gt_lint.py ~/my-vault --queue ~/my-vault/review-queue.md
```

`gt_tasks.py` is the exception: it ships **in the vault**, not the plugin, at `Projects/golden-thread/tools/`. It regenerates `TASKS.md` from every project's `## Tasks` section.

```
python3 ~/my-vault/Projects/golden-thread/tools/gt_tasks.py
```

It infers the vault from its own location, so it needs no arguments; `--vault` is there to override that. **Re-run it before reading `TASKS.md`** — project priority is computed against the clock (stale-P1 ageing, deadline windows, `pp_escalate`), so the ranking changes with time even when no file has changed. `TASKS.md` is generated and must never be hand-edited.

Troubleshooting

"vault-config.json not found" — run `/gt:gt-init`.

A rule in `CLAUDE.md` seems ignored — it is read at session start. Added mid-session, it applies from the *next* session. Same for hooks in `settings.json`: open `/hooks` once or restart.

Memory files aren't being read — that is intended. `gt-open` reads `MEMORY.md` and loads files on demand. Ask for one by name.

A memory file is never offered — run `/gt:gt-lint`; it is probably `memory-unlisted`, so nothing knows it exists.

`/gt:gt-query` can't find a page you added — likely `index-gap` .

Lint suppression isn't working — paths are matched case-insensitively against the finding path; make sure you used the path as the finding reports it.

"conflict: vault-config.json points to a different vault" — run

`vault_init.py connect --vault <new-path>` .